ПРАКТИЧНА РОБОТА №3

Тема: Локальний стан, ефекти, посилання та мемоізація в React
Мета: Навчитися керувати станом через useState, запускати й очищувати побічний ефект через useEffect, зберігати попереднє значення через useRef та обчислювати фільтр через useMemo.

ХІД РОБОТИ

1. Для збереження вибраної категорії, обраних подій і повідомлення використано useState:

```
const [events, setEvents] = useState(initialEvents);
const [category, setCategory] = useState<Category>('Усі');
const [favoriteIds, setFavoriteIds] = useState<string[]>([]);
const [filterNotice, setFilterNotice] = useState('');
```

2. Для отримання списку подій відповідно до вибраної категорії використано useMemo. Якщо вибрано «Усі», відображаються всі події; інакше масив фільтрується за властивістю category. Залежності [category, events] враховують як зміну категорії, так і видалення події.

```
const visibleEvents = useMemo(() => {
  console.count('Обчислення фільтра');

  if (category === 'Усі') {
    return events;
  }

  return events.filter((event) => event.category === category);
}, [category, events]);
```

3. Для кожної картки EventCard передаються props isFavorite та onToggleFavorite. Стан обраного зберігається у favoriteIds, тому перемикання категорії не скидає обрані події. Функціональне оновлення гарантує використання попереднього стану.

```
const toggleFavorite = (id: string) => {
  setFavoriteIds((previous) =>
    previous.includes(id)
      ? previous.filter((favoriteId) => favoriteId !== id)
      : [...previous, id]
  );
};
```

4. За допомогою useRef зберігається попередня категорія, а useEffect реагує на її зміну. Повідомлення очищується через дві секунди. Функція clearTimeout у cleanup ефекту скасовує попередній таймер, якщо користувач швидко перемикає категорії.

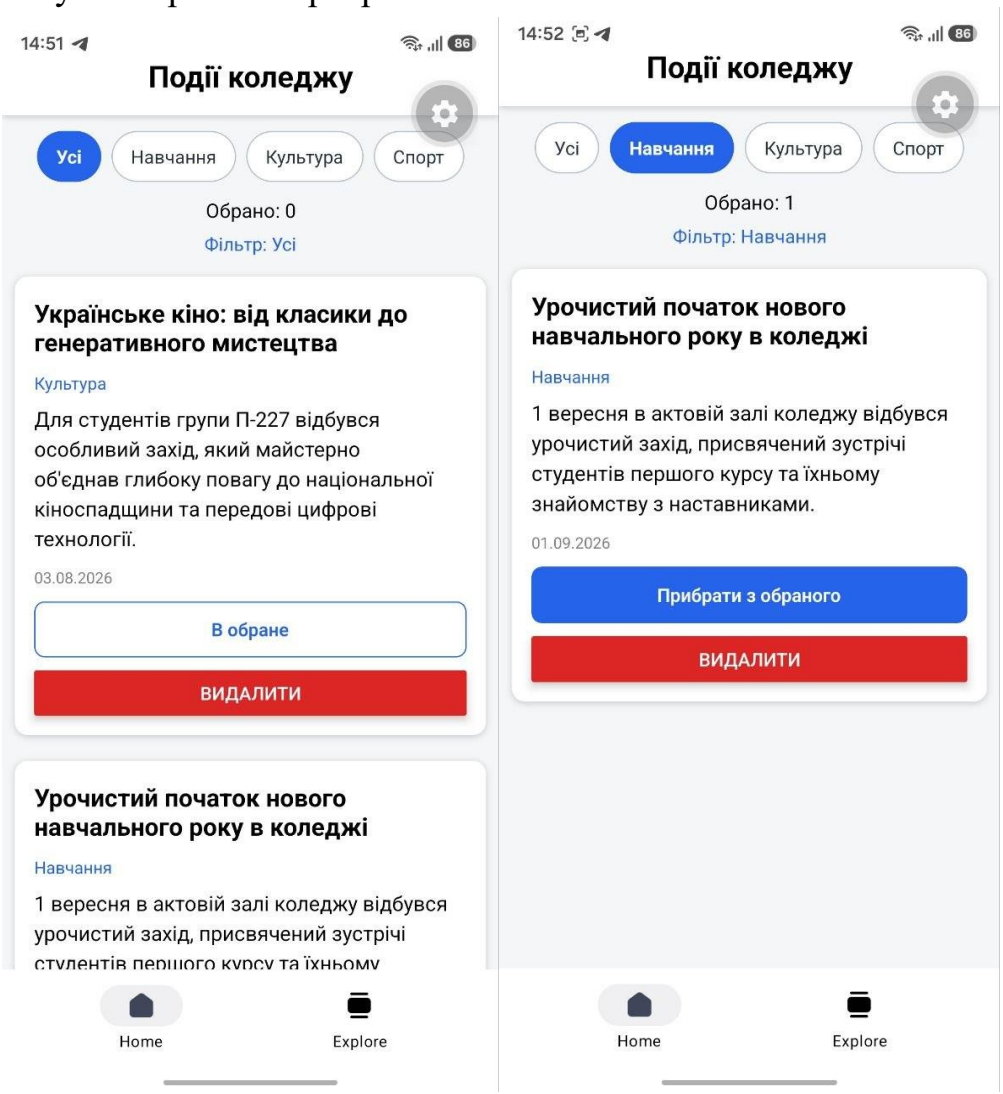
```
useEffect(() => {
  if (previousCategory.current === category) {
    return;
  }

  previousCategory.current = category;
  setFilterNotice(`Фільтр: ${category}`);

  const timer = setTimeout(() => {
    setFilterNotice('');
  }, 2000);

  return () => clearTimeout(timer);
}, [category]);
```

5. Результат роботи програми:



Контрольні питання

1. Локальна змінна не може замінити стан компонента, оскільки її зміна не викликає повторного рендерингу компонента. useState зберігає значення між рендерами та оновлює інтерфейс після зміни стану.
2. Функціональну форму оновлення стану (setState(previous => ...)) потрібно використовувати, коли нове значення залежить від попереднього стану. Це допомагає уникнути помилок під час кількох послідовних оновлень.
3. push() змінює початковий масив, не створюючи нового. React може не визначити зміну стану, тому інтерфейс не оновиться. Краще використовувати setState, створюючи новий масив, наприклад: setEvents(prev => [...prev, newEvent]).
4. useEffect потрібен для взаємодії із зовнішніми системами: таймерів, підписок, запитів до сервера або роботи з DOM. Ефект зайвий, якщо значення можна обчислити безпосередньо під час рендерингу або виконати дію в обробнику події, наприклад після натискання кнопки.
5. Функція очищення useEffect виконується перед повторним запуском ефекту, якщо змінилися його залежності, а також під час видалення компонента. Вона потрібна для очищення ресурсів, наприклад таймерів або підписок.
6. useState зберігає стан компонента, і його зміна викликає повторний рендеринг. useRef зберігає значення між рендерингами, але його зміна не викликає повторного рендерингу.
7. useMemo кешує результат обчислень, щоб уникнути зайвих повторних обчислень. Він призначений для підвищення продуктивності, а не для забезпечення правильності роботи програми, тому код має працювати правильно і без нього.